

# Splitting method and QSample

João Henrique Oliveira Fontes

May 21, 2026

## **Abstract**

# Contents

|       |                                                          |   |
|-------|----------------------------------------------------------|---|
| 0.1   | Splitting method overview . . . . .                      | 2 |
| 0.1.1 | Splitting method for estimating QEC thresholds . . . . . | 2 |
| 0.2   | The Splitting method . . . . .                           | 4 |
| 0.3   | Results . . . . .                                        | 6 |
| 0.4   | Conclusion . . . . .                                     | 8 |

## 0.1 Splitting method overview

The splitting method is based on an estimation method to calculate the free energy difference between two highly overlapping canonical ensembles[?]. Consider a system with a configuration space defined by coordinates  $\{q_1, \dots, q_N\}$ , on which we define a temperature-scaled energy function  $U_0(q_1, \dots, q_N) = \Phi_0(q_1, \dots, q_N)/k_B T$ . With this energy, we can build the partition function for the ensemble:

$$Z_0 = \int \exp[-U_0(q_1, \dots, q_N)] dq_1 \dots dq_N$$

An exact calculation of  $Z_0$  is often intractable, but given the same configuration space, the ratio between two partition functions  $Z_0/Z_1$  defined by energy functions  $U_0$  and  $U_1$  is tractable by Monte Carlo methods.

To calculate this ratio, we define a Metropolis function  $M(x) = \min\{1, e^{-x}\}$ , and notice that, for a given configuration of the ensemble, the two energy functions have the property

$$M(U_1 - U_0) \exp(-U_0) = M(U_0 - U_1) \exp(-U_1).$$

We can then integrate both sides over the configuration space and obtain:

$$\begin{aligned} \int M(U_1 - U_0) \exp(-U_0) dq_1 \dots dq_N &= \int M(U_0 - U_1) \exp(-U_1) dq_1 \dots dq_N = \\ Z_0 \frac{\int M(U_1 - U_0) \exp(-U_0) dq_1 \dots dq_N}{Z_0} &= Z_1 \frac{\int M(U_0 - U_1) \exp(-U_1) dq_1 \dots dq_N}{Z_1} = \\ Z_0 \langle M(U_1 - U_0) \rangle_0 &= Z_1 \langle M(U_0 - U_1) \rangle_1 \end{aligned} \quad (1)$$

So that we obtain the ratio as:

$$\frac{Z_0}{Z_1} = \frac{\langle M(U_0 - U_1) \rangle_1}{\langle M(U_1 - U_0) \rangle_0}. \quad (2)$$

The quantities on the right-hand side can be calculated by direct Monte Carlo sampling if  $|U_1 - U_0|$  is not too big. One way around if this difference is big is to define intermediate energy functions  $U_i$  to create a chain of highly overlapping ensembles between the two target ensembles. This is especially simple to do in our particular case of estimating the error rate of QEC codes, as we will see below.

### 0.1.1 Splitting method for estimating QEC thresholds

An application of the splitting method for the calculation of quantum error correction thresholds can be found in [?]. In the calculation of QEC thresholds, each energy function corresponds to a physical error rate  $p$  for which we want to find the logical error rate  $P(p)$ . We begin with a high enough  $p_0$  for which we can use regular Monte Carlo methods to obtain a reliable estimate  $P(p_0)$ , and obtain  $P(p)$  by multiplying over a chain of  $t$  physical error rates  $p_0 > p_1 > \dots > p_{t-1} > p$ :

$$P(p) = P(p_0) \cdot \frac{P(p_1)}{P(p_0)} \cdot \dots \cdot \frac{P(p)}{P(p_{t-1})}$$

If the circuit we are testing has  $N$  possible error locations, then a bitstring  $E$  of errors has a probability  $\pi_j = p_j^{|E|} (1 - p_j)^{N - |E|}$  to occur. Denoting the set of all bitstrings that lead to a logical fault by  $\mathcal{F}$ , the probability of a bitstring  $E$  given the presence of a logical fault is  $\pi_j(E|\mathcal{F}) = \pi_j(E)/P(p_j)$ . Then, any function of error configurations can be defined by:

$$\langle f \rangle_j = \sum_{E \in \mathcal{F}} \pi_j(E|\mathcal{F}) f(E)$$

Which we can estimate using  $T$  Monte Carlo samples by

$$\langle \hat{f} \rangle_j = \frac{1}{T} \sum_{\alpha=1}^T T f(E_\alpha)$$

provided that the distribution of  $E_\alpha$  approaches  $\pi_j(E|\mathcal{F})$ . We can obtain such a set  $\{E_\alpha\}_j$  by performing metropolis sampling over error configurations, beginning with a bitstring  $E_0$  that is known to cause a logical fault, and given a last known failing configuration  $E_i$ , adding a new bistring  $E_{i+1}$  that differs from  $E_i$  on a single error location with probability  $q = \min 1, \frac{\pi_j(E_i)}{\pi_j(E_{i+1})}$ . The  $E_i$ 's will form a Markov chain that will eventually become a stationary distribution on  $\mathcal{F}$  that approximates  $\pi_j(E|\mathcal{F})$ .

With two Markov chains  $\mathcal{M}_{j-1}$  and  $\mathcal{M}_j$  we can calculate the ratio  $r_j = \frac{P(p_j)}{P(p_{j-1})}$  by:

$$r_j = \frac{P(p_j)}{P(p_{j-1})} = c \cdot \frac{\langle g(c\pi_{j-1}/\pi_j) \rangle_{j-1}}{\langle g(c^{-1}\pi_j/\pi_{j-1}) \rangle_j}$$

for some constant  $c$  whose optimal value is  $r_j$ , which typically we arrive at by iterating the equation above 3 times. The algorithm is detailed below.

## 0.2 The Splitting method

---

**Algorithm 1** Splitting Method for Estimating  $P(p)$ 


---

```

1: Input: Probability sequence  $p_1, \dots, p_t = p$ , initial chain length  $T_{\text{init}}$ , initial failure probability  $P(p_1)$ .
2: Parameters: Scaling factor  $\lambda = 2$ , target relative error  $\epsilon = 0.25$ , function  $g(x) = \frac{1}{1+x}$ .
3: Output: Estimated failure probability  $\hat{P}(p)$  with relative error  $< \epsilon$ .
4: Initialize  $\hat{P} \leftarrow P(p_1)$ .
5: for  $j = 1$  to  $t$  do
    Gather Metropolis samples until desired precision reached
6:   Initialize Markov chain  $\mathcal{M}_j = [E_0]$ , where  $E_0$  is a known failing fault configuration.
7:   Set sequence length  $T_j \leftarrow T_{\text{init}}$ 
8:   Initialize  $\lambda' \leftarrow \lambda$ 
9:   while  $|\mathcal{M}_j| < T_j + 1$  do
10:    Extend  $\mathcal{M}_j$  using Metropolis updates (Algorithm 3 with  $p_j$ ) until length  $T_j + 1$ :  $\mathcal{M}_j = [E_0, E_1, \dots, E_{T_j}]$ 
11:    Compute sample estimates:  $\langle \hat{g}_{\pm} \rangle_j = \frac{1}{T_j} \sum_{\alpha=1}^{T_j} g\left(\frac{\pi_j(E_{\alpha})}{\pi_{j\pm 1}(E_{\alpha})}\right)$ 
12:    Compute sample variances:  $s_{\pm}^2 = \frac{1}{T_j-1} \sum_{\alpha=1}^{T_j} \left(g\left(\frac{\pi_j(E_{\alpha})}{\pi_{j\pm 1}(E_{\alpha})}\right) - \langle \hat{g}_{\pm} \rangle_j\right)^2$ 
13:    Estimate statistical error:  $\sigma = \max\left(\frac{s_+}{\langle \hat{g}_+ \rangle_j}, \frac{s_-}{\langle \hat{g}_- \rangle_j}\right) \frac{1}{\sqrt{T_j}}$ 
14:    Compute sample subset estimates (first half):  $\langle \hat{g}'_{\pm} \rangle_j = \frac{1}{\lfloor T_j/2 \rfloor} \sum_{\alpha=1}^{\lfloor T_j/2 \rfloor} g\left(\frac{\pi_j(E_{\alpha})}{\pi_{j\pm 1}(E_{\alpha})}\right)$ 
15:    Estimate mixing discrepancy:  $\Delta = \max\left(\frac{|\langle \hat{g}_+ \rangle_j - \langle \hat{g}'_+ \rangle_j|}{\langle \hat{g}_+ \rangle_j}, \frac{|\langle \hat{g}_- \rangle_j - \langle \hat{g}'_- \rangle_j|}{\langle \hat{g}_- \rangle_j}\right)$ 
16:    if  $(\sigma + \Delta) > \epsilon/\sqrt{t}$  then
17:       $T_j \leftarrow T_j + \lceil \lambda' \cdot T_{\text{init}} \rceil$ 
18:       $\lambda' \leftarrow \lambda \cdot \lambda'$ 
19:    end if
20:  end while
21:  Estimate ratio using Metropolis samples from current and previous round
22:  if  $j \geq 2$  then
23:    Initialize  $c \leftarrow 1$ 
24:    for  $i \in \{1, 2, 3\}$  do
25:      Compute ratio estimate:

$$\hat{r}_{j-1} = c \cdot \frac{\frac{1}{T_{j-1}} \sum_{\alpha=1}^{T_{j-1}} g\left(\frac{c \pi_{j-1}(E'_{\alpha})}{\pi_j(E'_{\alpha})}\right)}{\frac{1}{T_j} \sum_{\alpha=1}^{T_j} g\left(\frac{\pi_j(E_{\alpha})}{c \pi_{j-1}(E_{\alpha})}\right)}$$

26:       $c \leftarrow \hat{r}_{j-1}$ 
27:    end for
28:    Update failure probability estimate:

$$\hat{P} \leftarrow \hat{P} \cdot \hat{r}_{j-1}$$

29:  end if
30: end for
31: return  $\hat{P}$ 

```

---

### 0.3 Results

Comparison of the splitting method for the repetition code with different code distances and initial Markov chain length  $T_{\text{init}}$ . For code distance 5, the logical error rates for one realization of the splitting method for each  $T_{\text{init}} \in \{2, 4, \dots, 20\}$  are shown in Fig. 1.

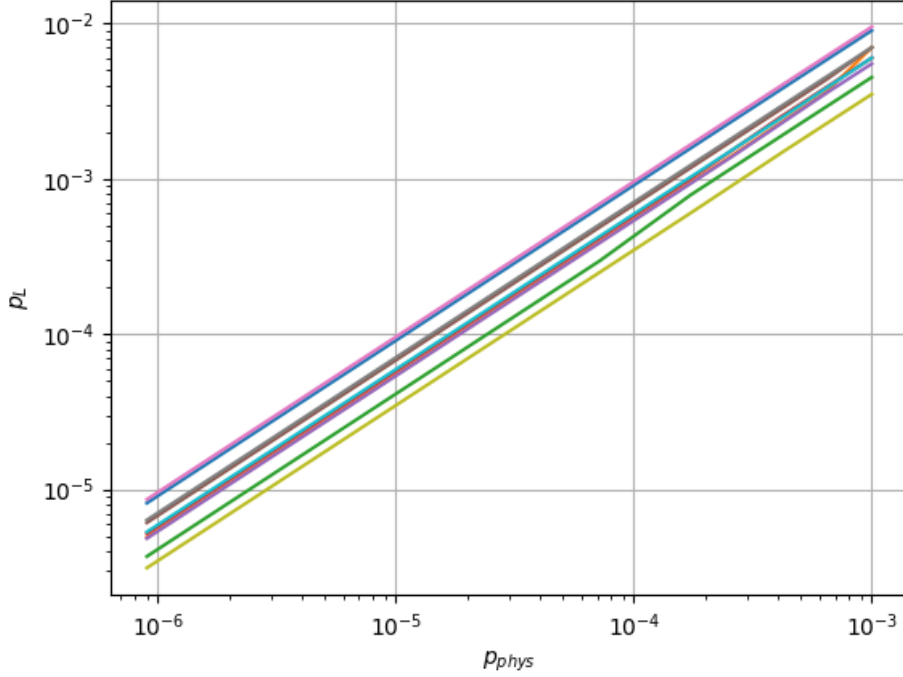


Figure 1

The difference between realizations can be attributed to different estimations of the initial logical error rate  $p_1$ , obtained using direct Monte Carlo sampling. The average and standard deviations of the logical error rates for multiple realizations at each  $T_{\text{init}}$ , corrected for differences in  $p_1$ , is shown in 2(a). The number of times the length of the Markov Chain  $T_j$  is updated is shown in (b). The total computing time grows linearly with the total length, as we can see in the comparison in (c) between the actual times and the expected times. The uncertainty of the logical error estimation at the lowest physical error rate is plotted as a function of  $T_{\text{init}}$  is plotted in (d). The uncertainty does not vary significantly as a function of  $T_{\text{init}}$ .

The same analysis was repeated for  $T_{\text{init}} \in \{10, 20, \dots, 100\}$ , as shown in Fig. 3:

And for  $T_{\text{init}} \in \{2, 4, \dots, 20\}$  with distance 11:

Which agrees with the results obtained with QSample when corrected for the initial sampling point:



Figure 2: Analysis for distance 5 repetition code and  $T_{\text{init}} \in \{2, 4, \dots, 20\}$ .

Figure 3: Analysis for distance 5 repetition code and  $T_{\text{init}} \in \{10, 20, \dots, 100\}$ .

Figure 4: Analysis for distance 11 repetition code and  $T_{\text{init}} \in \{2, 4, \dots, 20\}$ .

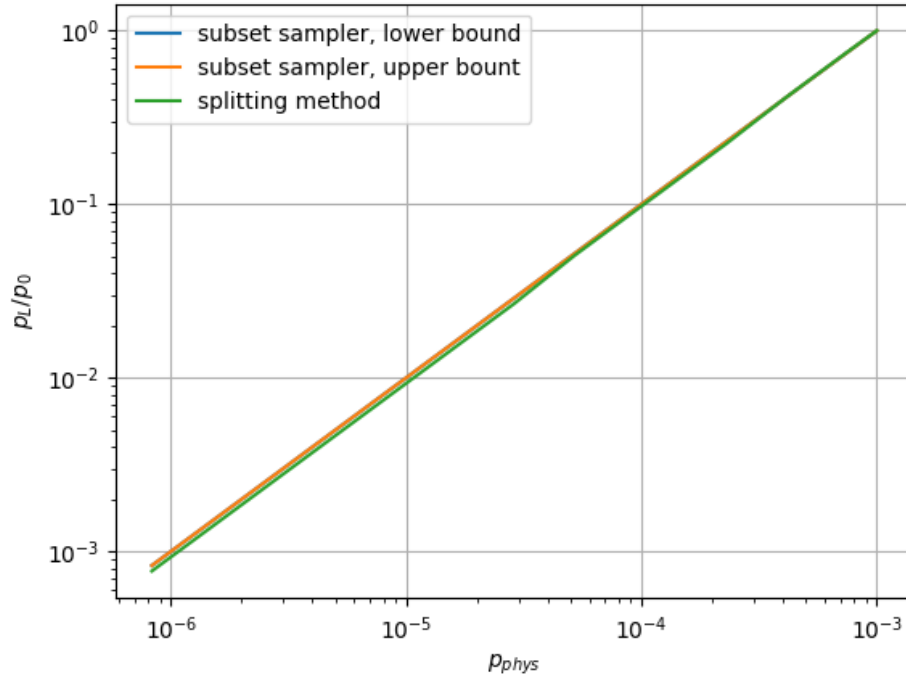


Figure 5: Comparison of logical error rates obtained by QSample and the splitting method for distance 11 repetition code.

## 0.4 Conclusion

In bocca lupo.